

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools.
(BL4,BL5)

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.
2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.
3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based

on overall link quality rather than just the shortest distance. Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.

Pseudocode: Network Performance Analysis

BEGIN

 // Input values

 total_data_received ← 8000 // bits

 time ← 4 // seconds

 total_packets_sent ← 200

 total_packets_received ← 180

 // Validation

 IF time = 0 THEN

 DISPLAY "Error: Time cannot be zero."

 STOP

 END IF

 IF total_packets_sent = 0 THEN

 DISPLAY "Error: Total packets sent cannot be zero."

 STOP

 END IF

 // Calculate throughput

 throughput ← total_data_received / time

 // Calculate packet loss

 packet_loss ← total_packets_sent - total_packets_received

 packet_loss_percentage ← (packet_loss / total_packets_sent) × 100

 // Display results

 DISPLAY "----- Network Performance Summary -----"

 DISPLAY "Total Data Received: ", total_data_received, " bits"

 DISPLAY "Time: ", time, " seconds"

 DISPLAY "Packets Sent: ", total_packets_sent

 DISPLAY "Packets Received: ", total_packets_received

 DISPLAY "Throughput: ", throughput, " bps"

```
DISPLAY "Packet Loss: ", packet_loss_percentage, "%"
```

```
// Classify network
```

```
IF throughput > 1500 AND packet_loss_percentage < 10 THEN
```

```
    classification ← "Efficient"
```

```
ELSE
```

```
    classification ← "Inefficient"
```

```
END IF
```

```
DISPLAY "Network Classification: ", classification
```

```
END
```

Expected Output

----- Network Performance Summary -----

Total Data Received: 8000 bits

Time: 4 seconds

Packets Sent: 200

Packets Received: 180

Throughput: 2000 bps

Packet Loss: 10%

Network Classification: Inefficient

Reason: Throughput is greater than 1500 bps, but packet loss is **10%**, which is not less than 10%.

Therefore, the network is classified as **Inefficient**.

2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.

Pseudocode: Network Bandwidth Monitoring and Management

BEGIN

// Store network links

links $\leftarrow$ ["Link1", "Link2", "Link3", "Link4"]

// Array to store bandwidth utilization

bandwidth_usage $\leftarrow$ [0, 0, 0, 0]

WHILE TRUE DO

 DISPLAY "----- Bandwidth Monitoring -----"

 // Monitor every network link

 FOR i $\leftarrow$ 0 TO length(links) - 1 DO

 // Collect current bandwidth utilization

 bandwidth_usage[i] $\leftarrow$ GET_BANDWIDTH_USAGE(links[i])

 DISPLAY links[i], " Utilization: ",

 bandwidth_usage[i], "%"

 // Check for overloaded link

 IF bandwidth_usage[i] > 80 THEN

 DISPLAY "Warning: ", links[i],

 " is overloaded."

 DISPLAY "Recommendation: Switch traffic to backup link."

 SWITCH_TRAFFIC_TO_BACKUP(links[i])

 ELSE

 DISPLAY links[i], " is operating normally."

 END IF

END FOR

// Wait before next monitoring cycle

WAIT 10 seconds

END WHILE

END

Example Output

----- Bandwidth Monitoring -----

Link1 Utilization: 65%

Link1 is operating normally.

Link2 Utilization: 85%

Warning: Link2 is overloaded.

Recommendation: Switch traffic to backup link.

Link3 Utilization: 72%

Link3 is operating normally.

Link4 Utilization: 91%

Warning: Link4 is overloaded.

Recommendation: Switch traffic to backup link.

Explanation

The algorithm continuously checks every network link at regular intervals and stores the bandwidth utilization values in an array. If utilization **exceeds 80%**, the link is identified as overloaded and traffic is recommended to be moved to a backup link.

This helps in **efficient bandwidth management** by detecting overloaded links early, distributing traffic to available backup links, and reducing the possibility of **network congestion, packet loss, and performance degradation**.

3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based on overall link quality rather than just the shortest distance. Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

Pseudocode: Dynamic Best-Path Selection Based on Link Quality

BEGIN

// Six branch offices

branches ← ["Branch1", "Branch2", "Branch3",
 "Branch4", "Branch5", "Branch6"]

// Store link information

bandwidth ← [100, 80, 120, 60, 100, 90] // Mbps

delay ← [10, 15, 8, 20, 12, 18] // ms

utilization ← [50, 65, 40, 85, 55, 70] // %

// Repeat monitoring periodically

WHILE TRUE DO

 DISPLAY "----- Network Monitoring -----"

 // Collect current link values

 FOR each link i DO

 bandwidth[i] ← GET_BANDWIDTH(i)

 delay[i] ← GET_DELAY(i)

 utilization[i] ← GET_UTILIZATION(i)

 DISPLAY "Link ", i

 DISPLAY "Bandwidth: ", bandwidth[i], " Mbps"

 DISPLAY "Delay: ", delay[i], " ms"

 DISPLAY "Utilization: ", utilization[i], "%"

 // Calculate link quality score

 score[i] ← (0.5 × bandwidth[i])

- $(0.3 \times \text{delay}[i])$
- $(0.2 \times \text{utilization}[i])$

```
// Check whether link is overloaded
IF utilization[i] > 80 THEN
    status[i] ← "OVERLOADED"
    DISPLAY "Warning: Link ", i, " is overloaded."
ELSE
    status[i] ← "AVAILABLE"
END IF

// Check whether link has failed
IF LINK_STATUS(i) = "FAILED" THEN
    status[i] ← "FAILED"
    DISPLAY "Alert: Link ", i, " has failed."
END IF

END FOR

// Select the best available path
best_score ←  $-\infty$ 
best_path ← NONE

FOR each path from Headquarters to every branch DO

    IF all links in path are AVAILABLE THEN

        path_score ← SUM(score of all links in path)

        IF path_score > best_score THEN
            best_score ← path_score
            best_path ← path
        END IF

    END IF

END FOR

// Select alternate path if required
IF best_path = NONE THEN
    DISPLAY "Alert: No suitable path available."
ELSE
    DISPLAY "Best Communication Path: ", best_path
    DISPLAY "Path Quality Score: ", best_score
END IF
```



```
// Wait before the next monitoring cycle
```

```
WAIT 10 seconds
```

```
END WHILE
```

```
END
```

Explanation

The algorithm continuously collects **bandwidth, delay, and utilization** for every network link and stores them in arrays. A **link quality score** is calculated using all three metrics, so the system does not select a path based only on distance.

A path with **higher bandwidth, lower delay, and lower utilization** receives a better score. If utilization exceeds **80%**, the link is marked as overloaded. If a link fails, it is removed from path selection. The algorithm then chooses the highest-scoring available alternate path and generates an alert.

Benefits

- **Reliability:** Continuously monitors link conditions.
- **Routing efficiency:** Selects paths using bandwidth, delay, and utilization rather than distance alone.
- **Fault tolerance:** Automatically avoids failed or overloaded links.
- **Adaptive routing:** Recalculates scores periodically as network conditions change.
- **Congestion prevention:** Detects heavily utilized links before they seriously affect communication.